

CMP514 – Computer Graphics and Image Processing Laboratory

B Tech CSE TY

Activity 5 – Line drawing using Bresenham's algorithm in C Graphics (graphics.h)

Aim: To implement and analyze Bresenham's line-drawing algorithm, comparing the rounding mechanisms, loop structures, and execution safety under edge cases.

Mathematical Foundations & Algorithms

Bresenham's Algorithm

Loop Bounds: $i = 0$ to $i \leq \text{length}$

1. Read the input pair of two end points (X_1, Y_1) and (X_2, Y_2)
Store left point in X_1, Y_1
2. Plot the first point be (X_1, Y_1)
3. Determine the initial value of the decision parameter by calculating the constants dx , dy , $2 \cdot dy$ and $2(dy - dx)$ as
$$dx = |x_2 - x_1|$$
$$dy = |y_2 - y_1|$$
$$D_0 = 2dy - dx$$
4. For each X_k , conduct the following test, starting from $k = 0$
If $D_k < 0$
 Plot next point at (X_{k+1}, Y_k) and $D_{k+1} = D_k + 2dy$
Else
 Plot next point at (X_{k+1}, Y_{k+1}) and $D_{k+1} = D_k + 2dy - 2dx$
 (update $Y_k = Y_{k+1}$) (update $X_k = X_{k+1}$)
5. Repeat step (4) dx times

Step 2: Visualization verification

Plot the lines using both variants – use different colors to plot these lines and observe the non-overlapping pixels.

Step 3: Benchmarking

While Bresenham's algorithms has a linear **time complexity of $O(N)$** relative to the number of pixels drawn, the performance differs at the microarchitecture level.

To benchmark this execution difference, we introduce a performance stress test that measures system execution clock cycles while plotting a dataset of **1,000 distinct lines**.

Metric Parameter	Variant 1 (Basic Rounded)	Variant 2 (Sign-Bias Optimized)	Bresenham's Algorithm
Total lines processed	1000 Lines	1000 Lines	1000 Lines
Total microseconds	<i>[Enter Log]</i>	<i>[Enter Log]</i>	<i>[Enter Log]</i>
Average time per line	<i>[Total Time / 1000]</i>	<i>[Total Time / 1000]</i>	<i>[Total Time / 1000]</i>
Calculated speed factor increase	Baseline (1.0)	<i>[Time_{V1} / Time_{V2}]</i>	<i>[Time_{V2} / Time_{VB}]</i>

Step 4: Programming Challenge

To deepen your understanding of structural decomposition, you will break away from incremental stepping and implement line drawing using alternative computational structures.

The Recursive Bresenham's Approach

Instead of updating an iterative loop counter, a recursive line drawer tracks line segments by passing the fractional floating-point variables down a call stack until it crosses the structural boundary limit.

- **Base Case:** If the structural loop depth matches or exceeds the required coordinate step bounds, stop execution and unwind the stack.
- **Recursive Step:** Plot the current coordinates using the chosen rounding method, apply the calculated increments directly to the parameters, and pass the updated states into the next recursive functional branch step.

The Divide-and-Conquer Approach (Midpoint Subdivision)

Instead of walking sequentially from start to end, a divide-and-conquer strategy bisects space. It finds the structural midpoint between two target endpoints, plots that midpoint pixel, and splits the line segment into two independent sub-segments.

- **Base Case:** If the distance between two coordinate inputs drops below a single pixel radius (e.g., $\max(|x_2 - x_1|, |y_2 - y_1|) \leq 1$), plot the endpoints and exit.
- **Conquer Step:** Calculate the geometric midpoint coordinates:
 $x_{mid} = (x_1 + x_2) / 2$, and $y_{mid} = (y_1 + y_2) / 2$
- **Divide Step:** Concurrently dispatch two independent sub-tasks to process the left half and the right half of the line:
 1. `drawSubdivision(x1, y1, x_mid, y_mid)`
 2. `drawSubdivision(x_mid, y_mid, x2, y2)`

Step 5 Architectural Comparison Matrix

Fill out the performance profile tracking table below after executing all four methods.

Algorithm	Algorithm Paradigm	Time Complexity	Memory/Stack Overhead	Risk Factors / Structural Limits
DDA	Iterative basic rounded	$O(N)$	$O(1)$ (Constant)	High precision drop error, Division by Zero
	Iterative sign-bias optimized	$O(N)$	$O(1)$ (Constant)	None (Robust Baseline)
	Recursive state-step	$O(N)$	[To deduce]	Stack Overflow on extremely long lines
	Divide-and-Conquer (subdivision)	[To deduce]	[To deduce]	Duplicate pixel rendering overhead
Bresenham	Iterative	$O(N)$	$O(1)$ (Constant)	[To deduce]
	Recursive state-step	$O(N)$	[To deduce]	[To deduce]
	Divide-and-Conquer (subdivision)	[To deduce]	[To deduce]	[To deduce]